EDUCAÇÃO A DISTÂNCIA

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Unidade 3 | Semana 6

Memória cache e integração dos componentes

INSTITUTO FEDERAL
Sudeste de Minas Gerais
Educação a Distância
Pró-reitoria de Ensino

Curso Técnico em Desenvolvimento de Sistemas

Professor: Jeronimo Costa Penha

Apostila V1 | 2026

Apresentação

Na Semana 5, estudamos a hierarquia de memória e os caminhos usados pelos dispositivos de entrada e saída. Nesta semana, vamos observar mais de perto um dos níveis que aproxima os dados do processador: a memória cache.

A cache guarda cópias de blocos que podem ser necessários em breve. Quando o dado procurado está nela, ocorre um acerto (hit); quando não está, ocorre uma falta (miss), e o sistema precisa buscá-lo em um nível inferior. A organização da cache determina onde cada bloco pode ficar e como outro bloco é escolhido quando há substituição.

Também acompanharemos o percurso de instruções e dados entre processador, cache, RAM, armazenamento e dispositivos. Assim, será possível relacionar os mecanismos de hardware ao comportamento de programas usados no Desenvolvimento de Sistemas.

Ideia central: a cache não aumenta a capacidade da RAM nem substitui o armazenamento; ela reduz o tempo médio de acesso quando os programas apresentam padrões de reutilização e proximidade.

Objetivos de aprendizagem

- Explicar a função da cache e relacioná-la à localidade temporal e espacial.
- Diferenciar acerto, falta, linha, bloco, conjunto, tag e deslocamento.
- Comparar mapeamento direto, totalmente associativo e associativo por conjuntos.
- Descrever políticas de substituição e noções introdutórias de escrita.
- Interpretar taxa de acerto e estimar o tempo médio de acesso em um exemplo simples.
- Descrever a integração entre CPU, cache, RAM, armazenamento e entrada e saída.
- Relacionar padrões de acesso de programas à localidade e ao desempenho.

Roteiro de estudos

1. Leia as seções 1 a 4 e acompanhe os exemplos de hit, miss e campos de endereço.
2. Assista ao vídeo sobre funcionamento e organização da cache.
3. Estude as seções 5 a 9, comparando os tipos de mapeamento e substituição.
4. Assista ao vídeo sobre integração e padrões de acesso em programas.
5. Resolva os exercícios, responda ao questionário e consulte os feedbacks.

Atividade	Tempo estimado
Leitura da apostila	1h30
Dois vídeos	25 min
Exemplos e exercícios	55 min
Questionário formativo	35 min
Revisão dos conceitos	1h05

1. Por que usar memória cache

A CPU pode executar instruções rapidamente, enquanto a memória principal tem um tempo de acesso maior do que os registradores internos do processador. Se a CPU precisasse esperar pela RAM em todos os acessos, parte de sua capacidade de processamento ficaria ociosa.

A cache reduz essa diferença mantendo, perto do processador, cópias de blocos trazidos de níveis inferiores. Ela é menor e mais rápida que a RAM. O sistema tenta guardar os blocos mais prováveis de serem usados novamente, sem exigir que o programador controle cada transferência diretamente.



Figura 1 — A cache faz parte da hierarquia que aproxima dados do processador.

Localidade temporal

Se um dado ou uma instrução foi usado há pouco, há boa chance de ser usado de novo em breve. Uma variável de contador atualizada em cada repetição de um laço, por exemplo, costuma ser lida e escrita muitas vezes durante a execução desse laço.

Localidade espacial

Se um endereço foi acessado, posições próximas também podem ser acessadas em seguida. Ao percorrer um vetor elemento por elemento, o programa tende a visitar endereços vizinhos. Por isso, uma falta pode levar para a cache um bloco com mais dados do que aquele solicitado naquele instante.

Padrão do programa	Tipo de localidade	Exemplo
Reutiliza a mesma variável em um laço	Temporal	Contador ou acumulador
Percorre posições consecutivas	Espacial	Leitura sequencial de vetor
Reutiliza uma função executada várias vezes	Temporal	Rotina chamada dentro de um ciclo

A cache funciona bem quando o padrão de acesso apresenta localidade. Ela não garante que todo acesso será rápido: a taxa de acerto depende do programa, da organização da cache e do conjunto de dados.

2. Acertos, faltas e tempo de acesso

Resultado	O que acontece	Consequência
Acerto (hit)	O bloco que contém o dado está na cache.	O dado pode ser entregue rapidamente à CPU.
Falta (miss)	O bloco não está na cache.	O sistema consulta um nível inferior, normalmente a RAM.

Após uma falta, o bloco é trazido do nível inferior e colocado na cache, se a política adotada permitir. A CPU recebe o dado solicitado e a execução continua. A primeira vez que um bloco é acessado costuma causar falta compulsória: ainda não havia cópia na cache.

Taxas de acerto e de falta

A taxa de acerto é a proporção dos acessos atendidos pela cache. A taxa de falta é a proporção que precisa consultar um nível inferior. Em uma contagem simples, as duas somam 100%.

```
taxa de acerto = acessos atendidos pela cache / total de acessos
taxa de falta  = acessos que falharam na cache / total de acessos
taxa de acerto + taxa de falta = 1 (ou 100%)
```

Exemplo de cálculo

Um programa realiza 1 000 acessos à memória. Se 920 são atendidos pela cache, a taxa de acerto é 920/1 000 = 0,92, ou 92%. Os 80 acessos restantes correspondem a uma taxa de falta de 8%.

Estimativa do tempo médio

Em um modelo introdutório, o tempo médio de acesso pode ser estimado como tempo de acerto + (taxa de falta × penalidade da falta). Aqui, a penalidade é o tempo adicional causado pela consulta ao nível inferior, além do tempo de acerto.

```
tempo médio ≈ tempo de acerto + taxa de falta × penalidade da falta
```

Exemplo: com hit de 2 ns, taxa de falta de 5% e penalidade adicional de 50 ns, a estimativa é 2 ns + 0,05 × 50 ns = 4,5 ns. O resultado é uma média: alguns acessos levam perto de 2 ns, enquanto os que falham demoram mais.

Em máquinas reais podem existir várias caches, sobreposição de operações e outros efeitos. A fórmula serve para compreender como a taxa de falta e o custo de uma falta influenciam o desempenho.

3. Blocos, linhas e campos de endereço

A memória principal é vista como uma sequência de endereços. A cache é organizada em linhas. Cada linha pode guardar um bloco trazido da memória, além de informações que ajudam a identificar de onde veio aquele bloco.

Campo	Função
Dados	Bytes do bloco copiado para a cache.
Tag (etiqueta)	Identifica qual bloco da memória está guardado na linha.
Índice / conjunto	Indica a linha ou o conjunto que deve ser consultado, conforme o mapeamento.
Deslocamento (offset)	Seleciona o byte ou a palavra dentro do bloco.
Bit válido	Indica se a linha contém um bloco utilizável.
Bit sujo (dirty), quando usado	Indica que o conteúdo foi alterado e ainda precisa ser escrito na RAM.

Em um endereço dividido em campos, o deslocamento identifica uma posição dentro do bloco. O índice localiza onde procurar. A tag é comparada com a armazenada na linha para confirmar que o bloco desejado está presente.

Exemplo de divisão de endereço

Considere endereços de 16 bits, blocos de 16 bytes e uma cache mapeada diretamente com 8 linhas. Como 16 bytes = 2^4, são necessários 4 bits de deslocamento. Oito linhas = 2^3, então 3 bits identificam a linha. Sobram 16 − 4 − 3 = 9 bits para a tag.



Figura 2 — Exemplo de campos em um endereço de 16 bits.

Para o endereço hexadecimal 0x1234: o deslocamento é 0x4 (os 4 bits menos significativos); o número do bloco é 0x123; o índice é 3 (0x123 módulo 8); e a tag é 0x24 (0x123 dividido por 8, descartando o resto). A cache compara essa tag com a linha de índice 3.

Esse exemplo ilustra a lógica do mapeamento direto. Em caches associativas, o índice pode identificar um conjunto, e a tag é comparada com as linhas daquele conjunto.

4. Mapeamento direto

No mapeamento direto, cada bloco da memória tem exatamente uma linha possível na cache. O cálculo didático mais comum usa: índice da linha = número do bloco módulo número de linhas da cache.

Exemplo: 16 blocos de memória e 4 linhas de cache

Linha da cache	Blocos de memória que podem ocupá-la
Linha 0	0, 4, 8, 12
Linha 1	1, 5, 9, 13
Linha 2	2, 6, 10, 14
Linha 3	3, 7, 11, 15

Os blocos 1, 5, 9 e 13 competem pela linha 1. Se o programa alternar continuamente entre os blocos 1 e 5, cada acesso pode substituir o anterior e provocar faltas repetidas, embora outras linhas estejam livres. Esse efeito é chamado de conflito de mapeamento.

Vantagens e limites

Aspecto	Descrição
Vantagem	Organização simples: o hardware sabe exatamente qual linha verificar.
Limite	Dois blocos que mapeiam para a mesma linha não podem permanecer juntos na cache.
Substituição	A posição é determinada pelo índice; não há escolha entre várias linhas candidatas.

O mapeamento direto tende a ser simples e rápido de verificar. Porém, a rapidez de consulta não impede conflitos quando o padrão de endereços concentra acessos nas mesmas linhas.

5. Cache totalmente associativa e associativa por conjuntos

Totalmente associativa

Em uma cache totalmente associativa, um bloco pode ser colocado em qualquer linha. O sistema compara a tag procurada com as tags válidas de todas as linhas. Há flexibilidade para evitar vários conflitos de índice, mas a busca e o controle do hardware são mais complexos.

Associativa por conjuntos

A cache é dividida em conjuntos, cada um com mais de uma linha, chamadas vias. O endereço seleciona um conjunto; dentro dele, o bloco pode ocupar qualquer via disponível. Essa organização combina uma busca restrita a um conjunto com mais opções de posicionamento do que o mapeamento direto.

Organização	Onde o bloco pode ficar?	O que é comparado?
Direta	Uma linha determinada pelo índice	Uma tag
Totalmente associativa	Qualquer linha da cache	Tags de todas as linhas
Por conjuntos	Qualquer via de um conjunto	Tags das vias do conjunto

Exemplo: cache de 4 linhas, associativa de 2 vias

Quatro linhas organizadas em 2 vias formam 2 conjuntos, cada um com 2 linhas. Usando o mesmo exemplo de 16 blocos, o conjunto pode ser calculado por número do bloco módulo 2. Blocos pares vão para o conjunto 0; blocos ímpares, para o conjunto 1. Dentro do conjunto, podem ocupar qualquer uma das duas vias.

Conjunto	Blocos possíveis	Espaço disponível
Conjunto 0	0, 2, 4, 6, 8, 10, 12, 14	Duas vias
Conjunto 1	1, 3, 5, 7, 9, 11, 13, 15	Duas vias

Os blocos 1 e 5 podem coexistir no conjunto 1, pois há duas vias. Se o programa alternar entre 1, 5 e 9, os três disputam o mesmo conjunto: como ele só tem duas vias, será necessário substituir algum bloco.

Comparação visual

Mais simples de localizar	Mais flexível para posicionar
Direta → associativa por conjuntos → totalmente associativa	Direta ← associativa por conjuntos ← totalmente associativa

Essa comparação é conceitual: projetos reais equilibram tempo de acesso, custo, consumo, área do chip e taxa de falta. Associatividade maior não significa automaticamente que todo acesso será mais rápido.

6. Políticas de substituição

Quando ocorre uma falta e todas as posições candidatas estão ocupadas, é preciso escolher um bloco para remover. Essa decisão é necessária em caches totalmente associativas e associativas por conjuntos. Em uma cache direta, o índice já determina a única linha que pode ser substituída.

Política	Regra simplificada	Observação
FIFO	Remove o bloco que está há mais tempo na cache.	Não considera se ele foi acessado recentemente.
LRU	Remove o bloco que ficou mais tempo sem ser acessado.	Procura aproveitar a localidade temporal.
Aleatória	Escolhe uma linha candidata ao acaso.	É simples e pode evitar alguns padrões ruins.

Exemplo com duas vias

Imagine um conjunto com dois blocos, A e B. A sequência de acessos foi A, B, A. Depois ocorre uma falta para C. Pela regra LRU, B é o menos recentemente usado e seria substituído. Pela regra FIFO, A seria removido se A tiver entrado antes de B, mesmo que tenha acabado de ser acessado.

Não existe uma política universalmente melhor

A eficiência depende do padrão de acessos. Uma política precisa acompanhar algum estado e usar recursos do hardware; a escolha real busca um equilíbrio entre custo e desempenho. Além disso, algumas implementações usam aproximações de LRU em vez de manter uma ordem exata de todos os acessos.

Importante: a substituição acontece entre as posições candidatas ao bloco. Em uma cache por conjuntos, o bloco de outro conjunto não precisa ser removido para abrir espaço naquele conjunto.

7. O que acontece quando a CPU altera um dado

Uma leitura procura trazer dados para perto da CPU. Uma escrita precisa manter uma política para que o valor alterado seja encontrado de forma consistente nos níveis da hierarquia. Existem diferentes estratégias; aqui vamos reconhecer duas decisões comuns.

Decisão	Estratégia	Ideia principal
Quando escrever na RAM?	Write-through	A escrita atualiza a cache e também é encaminhada à RAM.
Quando escrever na RAM?	Write-back	A linha é alterada na cache; a RAM é atualizada quando a linha suja é substituída ou em outro momento definido.
Falta durante escrita?	Write-allocate	Traz o bloco para a cache e então realiza a escrita nela.
Falta durante escrita?	No-write-allocate	Encaminha a escrita ao nível inferior sem carregar obrigatoriamente o bloco para a cache.

No write-back, um bit sujo pode indicar que a cópia na cache foi modificada e ainda difere da RAM. Antes de descartar essa linha, o sistema precisa preservar a alteração no nível apropriado. Isso reduz certas escritas imediatas, mas exige controle adicional.

Analogia cuidadosa

Pense na cache como uma cópia de trabalho próxima da CPU. Se a cópia mudar, o sistema precisa saber quando e como atualizar o nível inferior. A analogia ajuda a visualizar a ideia, mas o hardware implementa regras de consistência e sincronização que não se resumem a uma cópia manual.

A escolha das políticas depende do tipo de acesso e do projeto do processador. Para programação comum, compiladores, processador e sistema operacional geralmente cuidam desses detalhes; eles são úteis para compreender desempenho e depuração de baixo nível.

8. Caminho dos dados entre os componentes

A CPU executa instruções e precisa obter tanto as instruções quanto os dados necessários a cada etapa. A cache pode atender parte dessas solicitações. Se houver falta, um nível inferior fornece o bloco. O processador pode manter caches em mais de um nível, como L1, L2 e L3; detalhes e arranjos variam entre modelos.



Figura 3 — Caminho simplificado quando o dado é encontrado na cache.

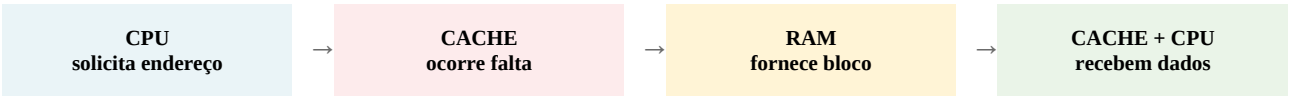


Figura 4 — Em uma falta, a solicitação alcança um nível inferior antes de prosseguir.

Integração com entrada e saída

Uma aplicação pode solicitar a leitura de um arquivo. O sistema operacional verifica buffers e cache de arquivos; se os dados precisarem vir do SSD, um driver programa o controlador. Em transferências maiores, DMA pode mover blocos entre o dispositivo e a RAM. Uma interrupção pode indicar a conclusão. Depois, a CPU processa os dados, buscando instruções e valores conforme o mecanismo de cache do processador.

Componente	Papel no fluxo
Aplicação e sistema operacional	Solicitam a operação e organizam buffers e permissões.
Driver e controlador	Traduzem a solicitação para comandos do dispositivo.
SSD / rede / periférico	Fornecem ou recebem os dados.
DMA e RAM	Podem participar da transferência e armazenar os blocos em uso.
CPU e cache	Executam instruções e acessam dados próximos ao processador.

Nem todo dado de entrada e saída segue exatamente o mesmo caminho. A sequência depende do dispositivo, do sistema operacional, dos controladores e da arquitetura. O diagrama apresenta as responsabilidades em nível introdutório.

9. Cache e Desenvolvimento de Sistemas

A cache é gerenciada principalmente pelo hardware, mas o padrão de acesso de um programa influencia quantas vezes os dados são encontrados nela. Uma estrutura de dados compacta e acessada de modo previsível pode aproveitar a localidade melhor do que acessos espalhados e com pouca reutilização.

Percurso sequencial e percurso com saltos

```
# acesso consecutivo: tende a aproveitar localidade espacial
para i de 0 até tamanho - 1:
    processar(vetor[i])

# acesso espaçado: pode visitar menos dados de cada bloco
para i de 0 até tamanho - 1, passo 16:
    processar(vetor[i])
```

O segundo percurso não é necessariamente mais lento em toda máquina ou aplicação; o resultado depende do tamanho dos dados, do compilador, da estrutura usada e de outras operações. O exemplo mostra como o espaçamento pode alterar o aproveitamento de cada bloco trazido para a cache.

Decisão de software	Relação possível com a memória
Percorrer uma sequência em ordem	Acessos vizinhos podem aproveitar o mesmo bloco.
Reutilizar dados recentemente lidos	A localidade temporal pode elevar a taxa de acerto.
Acessar nós espalhados em uma estrutura encadeada	Pode haver pouca localidade espacial e mais faltas.
Processar conjuntos muito grandes	O conjunto ativo pode ultrapassar a capacidade da cache.

Boas práticas

- Escolha estruturas de dados que atendam aos requisitos de clareza, correção e desempenho.
- Meça o programa com dados representativos antes de otimizar.
- Observe tempo, uso de memória e padrão de acesso; não deduza desempenho apenas pela aparência do código.
- Evite micro-otimizações sem evidência: a hierarquia, o compilador e o sistema operacional influenciam o resultado.

Entender cache ajuda a formular hipóteses sobre desempenho. A confirmação vem de medições e testes adequados, não de uma regra isolada sobre qual estrutura é sempre mais rápida.

10. Exemplo integrado e dúvidas frequentes

Exemplo — somar os valores de um vetor

1. A aplicação inicia e suas instruções e dados ativos ficam disponíveis na memória principal, quando necessário.
2. A CPU busca instruções; o processador verifica suas caches e pode encontrar acertos ou faltas.
3. A primeira leitura de parte do vetor pode causar falta. Um bloco que contém a posição solicitada é trazido de um nível inferior.
4. As próximas posições podem estar no mesmo bloco, aproveitando localidade espacial.
5. O acumulador é reutilizado em cada passo, favorecendo localidade temporal.
6. Se o conjunto ativo for maior que a cache, haverá substituições; a RAM continua sendo a referência para os dados que não cabem na cache.

Afirmação	Correção
“Cache é uma memória permanente para meus arquivos.”	Não. A cache de CPU é volátil e guarda cópias temporárias de blocos.
“Um miss significa que a instrução falhou.”	Não. Miss significa que o dado ou instrução procurado não foi encontrado naquele nível de cache.
“Aumentar a cache sempre acelera tudo.”	Não necessariamente. Resultado depende do padrão de acesso, da latência, da organização e de outros limites.
“O programador escolhe manualmente toda linha usada.”	Em programas comuns, o hardware gerencia as caches automaticamente.
“A cache substitui a RAM.”	Não. A RAM mantém programas e dados ativos em uma capacidade muito maior.

11. Exercícios de consolidação

Parte A — Conceitos

1. Explique por que a cache é útil mesmo quando o computador já possui memória RAM.
2. Diferencie localidade temporal e localidade espacial com um exemplo de programa.
3. O que significam hit e miss em uma cache?
4. Explique a função da tag, do índice e do deslocamento em um endereço dividido em campos.

Parte B — Organização

1. Uma cache direta tem 4 linhas e a memória é dividida em blocos numerados. Em qual linha pode ficar o bloco 11? E o bloco 15?
2. Na mesma cache, por que os blocos 3 e 7 podem causar conflito?
3. Uma cache de 4 linhas é organizada em 2 conjuntos de 2 vias. Quantos conjuntos há e quantas linhas candidatas existem para um bloco selecionado?
4. Compare a flexibilidade de posicionamento do mapeamento direto e do totalmente associativo.

Parte C — Desempenho e integração

1. Em 2 000 acessos, 1 800 foram hits. Calcule as taxas de hit e miss.
2. Use hit de 3 ns, taxa de falta de 10% e penalidade adicional de 40 ns para estimar o tempo médio.
3. Em um conjunto de duas vias, a ordem de acesso foi A, B, A; depois ocorre falta para C. Qual bloco a política LRU escolheria substituir?
4. Descreva, em ordem, como um arquivo em um SSD pode chegar à aplicação e onde cache e RAM participam.

Desafio aplicado

Uma aplicação processa repetidamente uma tabela grande e apresenta tempo de execução irregular. Cite duas hipóteses relacionadas ao padrão de acesso ou à hierarquia de memória e explique como você coletaria evidências antes de alterar o programa.

12. Respostas comentadas

Item	Resposta esperada
A1	A cache é menor e mais rápida e mantém cópias de blocos prováveis de uso. Ela reduz a demora média, mas não substitui a capacidade da RAM.
A2	Temporal: reutilizar recentemente uma variável; espacial: percorrer posições vizinhas de um vetor.
A3	Hit: o bloco está no nível de cache consultado. Miss: não está e é necessário buscar em nível inferior.
A4	Tag identifica o bloco; índice escolhe linha ou conjunto; deslocamento seleciona posição dentro do bloco.
B1	11 mod 4 = 3; 15 mod 4 = 3. Ambos podem ocupar a linha 3.
B2	3 mod 4 = 3 e 7 mod 4 = 3; ambos mapeiam para a mesma linha.
B3	Há 2 conjuntos; um bloco mapeado para um conjunto pode ocupar qualquer uma das 2 vias desse conjunto.
B4	No direto, o bloco só tem uma linha possível; no totalmente associativo, pode ficar em qualquer linha.
C1	Hit = 1 800/2 000 = 90%; miss = 200/2 000 = 10%.
C2	3 ns + 0,10 × 40 ns = 7 ns.
C3	B é o menos recentemente usado, pois A foi acessado por último; LRU substituiria B.
C4	A aplicação solicita leitura; sistema verifica buffers/cache; driver controla o SSD; dados chegam à RAM, possivelmente com DMA; o sistema sinaliza conclusão e disponibiliza os bytes à aplicação. A CPU processa os dados usando seus níveis de cache conforme necessário.

Comentário para o desafio

Hipóteses possíveis: conjunto ativo maior que a cache; acesso com saltos ou dados espalhados; falta de reutilização; interferência de outras tarefas; gargalo fora da CPU. Evidências podem incluir medições repetidas, perfilador, comparação entre percursos sequenciais e espaçados e observação de tempo e uso de memória com o mesmo conjunto de dados. Primeiro formule e teste hipóteses; depois mude o código.

13. Resumo

- A cache mantém cópias temporárias de blocos para reduzir o tempo médio de acesso da CPU.
- Localidade temporal é reutilizar dados recentes; localidade espacial é acessar posições próximas.
- Hit encontra o bloco no nível consultado; miss exige buscar em um nível inferior.
- Linhas guardam dados e metadados como tag e bit válido; endereços podem ser divididos em tag, índice e deslocamento.
- No mapeamento direto, cada bloco tem uma linha possível; no associativo pode usar qualquer linha; por conjuntos pode ocupar uma via do conjunto indicado.
- FIFO, LRU e aleatória são exemplos de substituição quando há mais blocos candidatos do que posições disponíveis.
- Políticas de escrita definem quando alterações chegam à RAM e o que ocorre em uma falta de escrita.
- CPU, cache, RAM, controladores e dispositivos trabalham em conjunto; o caminho exato varia conforme o sistema.
- Padrões de acesso podem influenciar o aproveitamento da cache, mas medições devem confirmar qualquer hipótese de desempenho.

Glossário

Termo	Definição
Bloco	Unidade de dados movimentada entre níveis da hierarquia.
Cache line / linha	Posição da cache que armazena um bloco e informações de controle.
Conjunto e via	Conjunto é um grupo de linhas candidatas; via é uma linha dentro dele.
Hit / miss	Acerto / falta no nível de cache consultado.
Localidade	Tendência de reutilizar dados recentes ou acessar posições próximas.
Penalidade da falta	Tempo adicional para buscar dados em nível inferior.
Tag	Campo usado para identificar o bloco armazenado na linha ou conjunto.

Referências

PATTERSON, David A.; HENNESSY, John L. Organização e Projeto de Computadores: a interface hardware/software. Rio de Janeiro: Elsevier, 2017.

STALLINGS, William. Arquitetura e Organização de Computadores. São Paulo: Pearson, 2017.

TANENBAUM, Andrew S.; AUSTIN, Todd. Organização Estruturada de Computadores. São Paulo: Pearson, 2013.

MIT OPENCOURSEWARE. Computation Structures, Spring 2017. Cache memories, direct-mapped caches, block size, conflicts and associative caches. https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/pages/c14/c14s2/c14s2v6/ (acesso em 29 set. 2026).

Material elaborado para a disciplina Arquitetura e Organização de Computadores — IF Sudeste MG, 2026.